

Day 3 Learning

#Inheritance in Java

Inheritance is a fundamental concept in Object-Oriented Programming that allows a new class to base its definition on an existing class. This creates a hierarchy where a subclass (child) inherits the fields and methods of a superclass (parent).

Key Concepts

- Relationship: It represents an IS-A relationship. For example, a "Car" IS-A "Vehicle."
- Keywords: Use the "extends" keyword to create a subclass.
- Reusability: It allows you to reuse code from the parent class without rewriting it.
- Access: A subclass inherits all public and protected members. It does not inherit private members directly, though they can be accessed via public getter or setter methods.

Types of Inheritance in Java

1. Single Inheritance: A subclass inherits from one superclass.
2. Multilevel Inheritance: A subclass inherits from a parent class, which in turn inherits from another parent class (e.g., C extends B, and B extends A).
3. Hierarchical Inheritance: Multiple subclasses inherit from a single superclass.

Note: Java does not support Multiple Inheritance (one class extending multiple classes) to avoid complexity and ambiguity, such as the Diamond Problem.

#Inheritance Scenario-Based Questions and Answers:

Question 1: What is the execution order of constructors in an inheritance hierarchy?

Answer: When a subclass object is created, the constructor of the parent class is executed first, followed by the constructor of the child class. This happens because Java implicitly inserts a `super()` call as the first statement in a subclass constructor to ensure the parent state is initialized before the child state.

Question 2: Can a subclass directly access private members of its superclass?

Answer: No, private members are not inherited and cannot be accessed directly by a subclass. To interact with private fields of a parent class, the parent class must provide

public or protected getter and setter methods. This is a key part of maintaining encapsulation.

Question 3: Why does Java not support multiple inheritance with classes?

Answer: Java avoids multiple inheritance to prevent the Diamond Problem. This occurs when a subclass inherits from two parents that both have a method with the same signature. To avoid the ambiguity of which method to execute, Java restricts a class to extending only one superclass, though it allows implementing multiple interfaces.

Question 4: What is the difference between method overriding and method hiding in inheritance?

Answer: Method overriding applies to instance methods and is resolved at runtime based on the object type. Method hiding occurs when a subclass defines a static method with the same signature as a static method in the superclass. Unlike overriding, hiding is resolved at compile-time based on the reference type used to call the method.

Question 5: How can you prevent a class from being inherited or a method from being overridden?

Answer: To prevent a class from being inherited, you must declare it using the final keyword (e.g., public final class). If you want to allow inheritance but prevent a specific method from being redefined by a subclass, you declare that specific method as final.

Question 6: When should inheritance be avoided even if it is technically possible?

Answer: Inheritance should only be used when there is a clear IS-A relationship. If the relationship is actually HAS-A (for example, a House HAS-A Bathroom), you should use Composition instead of Inheritance. Using inheritance for a HAS-A relationship makes the code rigid and difficult to maintain.

#Polymorphism in Java

Polymorphism is the ability of an object to take on many forms. The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object. It allows for flexibility and the ability to define a single interface with multiple underlying implementations.

Types of Polymorphism

1. **Compile-time Polymorphism (Static Binding):** This is achieved through Method Overloading. The compiler determines which method to call based on the method signature at the time of compilation.
2. **Runtime Polymorphism (Dynamic Binding):** This is achieved through Method Overriding. The call to an overridden method is resolved at runtime based on the actual object type, not the reference type.

Method Overloading:

Method Overloading occurs when multiple methods in the same class have the same name but different parameter lists.

Rules for Overloading:

- Methods must have different parameters (number, type, or sequence).
- Changing only the return type is not sufficient for overloading and will result in a compile-time error.
- Access modifiers and exceptions can be different.

Method Overriding:

Method Overriding occurs when a subclass provides a specific implementation for a method that is already defined in its superclass.

Rules for Overriding:

- The method must have the same name and the same parameter list as in the parent class.
- The return type must be the same or a covariant (subtype) of the original return type.
- The access level cannot be more restrictive than the overridden method's access level (e.g., if the parent method is public, the child method cannot be private).
- Final, static, and private methods cannot be overridden.

Method Hiding:

Method Hiding occurs when a subclass defines a static method with the same signature as a static method in the superclass.

Key Characteristics of Method Hiding:

- It applies only to static methods.
- The method call is determined by the reference type at compile-time.

- If you use a Parent reference to point to a Child object and call a hidden static method, the Parent's version will execute.

#Interview Questions and Answers

Question 1: Can we overload a method by changing only its return type?

Answer: No. In Java, method signature consists only of the method name and parameter list. The compiler does not consider the return type when distinguishing between methods. If two methods have the same name and parameters but different return types, it will cause a compile-time error.

Question 2: Why is Method Overriding known as Runtime Polymorphism?

Answer: It is called Runtime Polymorphism because the Java Virtual Machine (JVM) determines which version of the method to execute (parent or child) during execution, based on the specific object being referred to, rather than the type of the reference variable.

Question 3: Can we override a static method?

Answer: No, static methods cannot be overridden. Static methods belong to the class, not the instance. If a subclass defines a static method with the same signature, it is Method Hiding, not overriding.

Question 4: What is the `@Override` annotation and is it mandatory?

Answer: The `@Override` annotation informs the compiler that the subclass is intending to override a method from the superclass. It is not mandatory, but it is highly recommended because it provides a compile-time check. If the method signature does not match exactly, the compiler will throw an error.

Question 5: Can a subclass override a method and throw a broader checked exception?

Answer: No. An overriding method can throw the same exception, a narrower exception, or no exception at all. However, it cannot throw a broader or new checked exception than the one declared in the superclass method.

Question 6: What happens if you try to override a method marked as final?

Answer: You will get a compile-time error. The "final" keyword is specifically used to prevent a method from being overridden by any subclass.

#Constructors in java (interview questions):

Question 1: Can a constructor be inherited?

Answer: No, constructors are not inherited in Java. Each class has its own constructors. However, when a subclass is instantiated, it automatically calls the constructor of its superclass (either implicitly or explicitly) to initialize the state of the parent part of the object.

Question 2: What happens if a Parent class has a parameterized constructor but no default (no-arg) constructor?

Answer: This will cause a compile-time error in the Child class unless the Child class explicitly calls the Parent's parameterized constructor using `super(args)`. By default, Java tries to insert `super()` (the no-arg call) in the child's constructor. If the parent doesn't have one, the code won't compile.

Question 3: Can we use "this()" and "super()" together in the same constructor?

Answer: No. Both `this()` (to call another constructor in the same class) and `super()` (to call a parent constructor) must be the very first statement in a constructor. Since there can only be one "first" statement, you cannot use both in the same constructor.

Question 4: Can a constructor be final, static, or abstract?

Answer: No.

- Final: Constructors are not inherited, so overriding them isn't possible anyway; `final` is irrelevant.
- Static: A constructor is used to initialize an object (instance). Static members belong to the class, not an object.
- Abstract: An abstract method has no implementation and must be overridden. Since constructors cannot be inherited or overridden, they cannot be abstract.

Question 5: What is the "Constructor Chaining" concept in Java?

Answer: Constructor chaining is the process of calling one constructor from another within the same class (using `this()`) or from the parent class (using `super()`). This ensures that all layers of an object's hierarchy are properly initialized in a specific sequence.

Question 6: If a constructor throws a checked exception, what must the subclass constructor do?

Answer: If a Parent class constructor declares a checked exception (e.g., `throws IOException`), every constructor in the Child class must also declare that same exception or a broader one. This is because the child constructor will always invoke the parent constructor, and therefore must handle or declare the potential exception.

Question 7: Can we call a subclass method inside a Parent constructor? Why is this risky?

Answer: Technically, yes, but it is a dangerous practice. If the Parent constructor calls a method that the Child class has overridden, the Child's version will run. However, at that moment, the Parent constructor hasn't finished, and the Child's instance variables haven't been initialized yet. This can lead to `NullPointerException` or unexpected behavior.

#Private Constructors in Java

A private constructor is a constructor that has the "private" access modifier. This prevents the class from being instantiated from any code outside of its own class body.

Purpose and Use Cases:

1. Singleton Design Pattern: This is the most common use. It ensures that only one instance of the class ever exists. The class manages its own instance and provides it through a public static method.
2. Utility Classes: Classes that contain only static utility methods (like `java.lang.Math`) use private constructors because there is no reason to ever create an object of that class.
3. Factory Methods: A class can use a private constructor to force users to create objects through specific static "factory" methods, allowing the class to control the instantiation logic.
4. Preventing Inheritance: Since a subclass must call a parent constructor, making the parent constructor private effectively prevents any other class from extending it.

Interview Questions: Private Constructors

Question 1: If a class has a private constructor, can we still create an object of that class?

Answer: Yes, but only from within the class itself. A static method inside the class can call the private constructor, create an object, and return it to an external caller.

Question 2: Can a class with only private constructors be inherited?

Answer: No. Every subclass constructor must call a constructor of its superclass (super()). If the superclass constructor is private, the subclass cannot access it, which causes a compile-time error.

Question 3: What is the difference between using a private constructor and the "final" keyword to prevent inheritance?

Answer: A "final" class is a direct way to stop inheritance while still allowing the class to be instantiated. A private constructor is primarily used to control instantiation; preventing inheritance is a side effect because the subclass cannot access the parent constructor.

Question 4: Can you have both a public and a private constructor in the same class?

Answer: Yes. This is known as constructor overloading. You might have a private constructor that contains the main initialization logic, which is then called by various public constructors using the this() keyword.

#Immutable classes in java:

In Java, an **Immutable Class** is a class whose state (the data stored in the object) cannot be modified after it is created. Once an instance is initialized, it remains constant for its entire lifetime.

The most famous example of an immutable class in Java is **String**, along with the wrapper classes like **Integer**, **Double**, and **Boolean**.

How to Create an Immutable Class

To make a class immutable, you must follow these five standard rules:

1. **Declare the class as final:** This prevents other classes from inheriting it and overriding methods to change the data.
2. **Make all fields private and final:**
 - **private** prevents direct access from outside.
 - **final** ensures that the variable can only be assigned once (during construction).
3. **No "Setter" methods:** Do not provide methods that modify the fields.

4. **Initialize through Constructor:** All fields should be set using a constructor that performs a "deep copy" if necessary.
5. **Perform Deep Copies in Getters:** If the class contains mutable objects (like a `Date` or a `List`), do not return the actual object reference. Instead, return a copy of it so the original data isn't modified from the outside.

Benefits of Immutability

- **Thread Safety:** Since the state never changes, multiple threads can access the object simultaneously without any risk of data corruption or synchronization issues.
- **Caching and Reuse:** You can safely cache immutable objects. This is why Java can use a "String Pool."
- **Security:** They are safe to use as keys in a `HashMap` or elements in a `HashSet` because their hash code will never change.

Interview Questions: Immutable Classes

Question 1: Why is it important to make the class `final`?

Answer: If the class is not `final`, a developer could create a subclass and override the "getter" methods to return different values, or add mutable state to the child class, which breaks the promise of immutability for the parent reference.

Question 2: If a class has all final fields, is it automatically immutable?

Answer: Not necessarily. If one of those final fields points to a mutable object (like an `ArrayList`), the *reference* cannot change, but the *contents* of the list can still be modified. You must ensure the contents are also protected.

Question 3: What is the difference between a "Shallow Copy" and a "Deep Copy" in this context?

Answer: A shallow copy just copies the reference to an object. A deep copy creates a completely new object with the same data. For immutability, you must use deep copies for any mutable fields so the internal state cannot be altered via an external reference.

Question 4: Are String objects truly immutable if I can do `str = str + " New";`?

Answer: Yes, they are still immutable. When you perform that operation, you aren't changing the original string. Instead, Java is creating a brand-new String object in memory and pointing the reference `str` to that new object. The original data remains unchanged.

#Coupling and Cohesion in Software Design

What should be followed: **"High Cohesion and Low Coupling"**

Cohesion:

Cohesion is the measure of how focused and related the internal elements of a module are. In a highly cohesive system, each class or module has a single, well-defined purpose.

Key Points:

- High Cohesion is the goal.
- It follows the Single Responsibility Principle (SRP).
- Low cohesion leads to "spaghetti code" where unrelated logic is mixed together, making it hard to debug and scale.

Coupling:

Coupling is the measure of dependency between different modules or classes. It describes how much one module knows about the inner workings of another.

Key Points:

- Low Coupling (Loose Coupling) is the goal.
- It is achieved through abstraction, interfaces, and dependency injection.
- High coupling (Tight Coupling) creates a fragile system where a small change in a parent class can break multiple dependent child classes.

Summary:

The ideal software architecture is one where modules are Highly Cohesive (do one thing well internally) and Loosely Coupled (don't depend heavily on other modules).

Interview Questions: Coupling and Cohesion

Question 1: Why is High Cohesion and Low Coupling preferred in Java?

Answer: High cohesion makes classes easier to maintain and reuse because they have a single focus. Low coupling makes the system flexible, allowing developers to change or replace one part of the system without affecting others, which reduces the risk of bugs during updates.

Question 2: How does an Interface help in achieving Low Coupling?

Answer: Interfaces allow a class to interact with another class without knowing its specific implementation. For example, if a class depends on a "Shape" interface, it doesn't care if the actual object is a "Circle" or a "Square." This decouples the caller from the specific implementation.

Question 3: Can you have a system with Low Cohesion but Low Coupling?

Answer: Yes, but it results in poor design. The code would be scattered across many modules with no clear purpose, even if those modules aren't heavily dependent on each other. It makes the logic very difficult to track and understand.

Question 4: What is a "God Object" in the context of Cohesion?

Answer: A "God Object" is a class that has very low cohesion because it has grown too large and performs too many different tasks. It is considered an anti-pattern because it is difficult to test, maintain, and violates the Single Responsibility Principle.

#SOLID Principles in OOPS:

1. **Single Responsibility Principle (SRP):** A class should have one, and only one, reason to change. This means a class should perform only one specific task. High cohesion is a direct result of following SRP.
2. **Open/Closed Principle (OCP):** Software entities should be open for extension but closed for modification. You should be able to add new functionality without altering existing code, usually achieved through interfaces and abstract classes.
3. **Liskov Substitution Principle (LSP):** Objects of a superclass should be replaceable with objects of its subclasses without breaking the application. Subclasses must fulfill the contract defined by the parent class.
4. **Interface Segregation Principle (ISP):** A client should never be forced to implement an interface that it does not use. Large interfaces should be split into smaller, more specific ones so that clients only know about the methods that are of interest to them.
5. **Dependency Inversion Principle (DIP):** Depend on abstractions, not on concretions. High-level modules should not depend on low-level modules; both should depend on interfaces. This reduces tight coupling between different layers of an application.

Interview Questions: SOLID Principles

Question 1: What is the main benefit of following SOLID principles?

Answer: The main benefits are improved code maintainability, scalability, and reusability. It reduces the "fragility" of the code, meaning changes in one part of the system are less likely to break other unrelated parts.

Question 2: How does the Open/Closed Principle relate to Polymorphism?

Answer: OCP is often implemented using Polymorphism. Instead of using "if-else" or "switch" blocks to check for types (which requires modification every time a new type is added), we use a common interface. New functionality is added by creating a new class that implements that interface (extension).

Question 3: Can you explain the "Square-Rectangle" problem in Liskov Substitution Principle?

Answer: If a Square extends Rectangle, it might break the logic. A Rectangle allows setting width and height independently. However, in a Square, setting the width also changes the height. If a piece of code expects a Rectangle and changes only the width, the Square will behave unexpectedly, violating LSP.

Question 4: What is the difference between ISP and SRP?

Answer: SRP is about the focus of a Class (internal logic). ISP is about the focus of an Interface (what is exposed to the client). While both aim for focus and simplicity, SRP handles how you build the class, and ISP handles how you design the API for others to use.